

Contents

1	Template	2			
2	Data Structures	2			
2.1	Segment Tree	2	3.10	Dijkstra	7
2.2	Merge Sort Tree	2	3.11	Dinic	7
2.3	Segment Tree Recursive	2	3.12	Euler Path Directed	8
2.4	Lazy	2	3.13	Euler Path Undirected	8
2.5	Persistent Segment Tree	3	3.14	Flood Fill	8
2.6	Dynamic Segment Tree	3	3.15	Floyd-Warshall	9
2.7	Sparse Table	3	3.16	Kruskal	9
2.8	Treap	3	3.17	MCMF	9
2.9	BIT	5	3.18	Prims	9
2.10	DSU	5	3.19	SCC Kosaraju	9
2.11	Trie	5	3.20	State Space Graph	10
3	Graphs	6	3.21	Topological Sort	10
3.1	2-SAT	6	4	Trees	10
3.2	Block Cut Tree	6	4.1	Centroid Decomposition	10
3.3	Bellman-Ford Cycle	6	4.2	Tree Diameter	10
3.4	Bellman-Ford Path	6	4.3	Euler Tour	10
3.5	BFS	6	4.4	Heavy Light Decomposition	10
3.6	Bipartite Check	7	4.5	LCA	11
3.7	Bridges	7	4.6	Min/Max Weight on Path	11
3.8	Bridge Tree	7	4.7	Intersection Path	11
3.9	DFS	7	4.8	Union Path	12
			4.9	Prufer Code Build	12
			4.10	Prufer Code Restore	12
			4.11	Tree Rerooting	12
			4.12	Tree Matching	13
			4.13	Rooted Tree Hashing	13
			4.14	Tree Isomorphism	13
5	Number Theory	13	6	Combinatorics	13
6	Combinatorics	13	6.1	nCr	13
7	Math	13	7	Math	13
7.1	FFT	13	7.1	FFT	13
7.2	FWDT	13	7.2	FWDT	13
7.3	Xor Basis	13	7.3	Xor Basis	13
7.4	Matrix	14	7.4	Matrix	14
8	Strings	14	8	Strings	14
8.1	Aho Corasick	14	8.1	Aho Corasick	14
8.2	Dynamic Aho Corasick	14	8.2	Dynamic Aho Corasick	14
8.3	Suffix Array	15	8.3	Suffix Array	15
8.4	Hashing	15	8.4	Hashing	15
9	Dp	15	9	Dp	15
9.1	Convex Hull Trick	15	9.1	Convex Hull Trick	15
9.2	Knuth Division	16	9.2	Knuth Division	16
9.3	Li Chao Tree	16	9.3	Li Chao Tree	16
10	Miscellaneous	16	10	Miscellaneous	16
10.1	Rnd template	16	10.1	Rnd template	16

1 Template

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 #define all(v) v.begin(),v.end()
6 #define rall(v) v.rbegin(),v.rend()
7 #define pb push_back
8 #define sz(a) int(a.size())
9 #define F first
10 #define S second
11 typedef long long ll;
12 typedef vector<int> vi;
13 typedef vector<ll> vll;
14 typedef pair<int,int> pii;
15 typedef pair<ll,ll> pll;
16 const int MOD = 1e9+7;
17 const int INF = INT_MAX;
18 const ll INF64 = LLONG_MAX;
19 const long double EPS = 1e-9;
20 const long double PI = acos(-1.0L);
21 void setIO(string p){
22     freopen((p + ".in").c_str(), "r", stdin);
23     freopen((p + ".out").c_str(), "w", stdout);
24 }
25
26 void solve(){
27
28 }
29
30
31 int main(){
32     ios_base::sync_with_stdio(0);
33     cin.tie(0);
34     cout.tie(0);
35     int tc = 1;
36     //cin >> tc;
37     for (int t = 1; t <= tc; t++)solve();
38 }

```

2 Data Structures

2.1 Segment Tree

```

1 struct Node{
2     ll val = 0;
3 };
4 Node operator+(Node a,Node b){return {a.val+b.val};}
5 class Segment_Tree{
6 public:
7     int n;
8     vector<Node> tree;
9
10    Segment_Tree(int x = 1e5+10){
11        n = x;
12        tree.resize(2*n+1);
13    }
14
15    void update(int p, ll val){
16        for(tree[p+n] = {val};p>1;p>>=1)
17            tree[p>>1] = tree[p]+tree[p^1];
18        //for (tree[p+n] = {val};p>>= 1;)t[p] = t[p<<1]+t[p<<1|1];
19    }
20    ll query(int l,int r){
21        Node s;
22        for(l+=n,r+=n ; l<r ; l>>=1 , r>>=1){
23            if(l&1)s = s+ tree[l++];

```

```

24            if(r&1)s = s+ tree[--r];
25        }
26        return s.val;
27    }
28 };

```

2.2 Merge Sort Tree

```

1 struct Node{
2     vi val;
3
4     int search(int x){
5         int l = 0, r = sz(val)-1;
6         while(l<=r){
7             int m = l+(r-l)/2;
8             if(val[m] < x)l =m+1;
9             else r = m-1;
10        }
11        return sz(val)-1;
12    }
13 };
14
15 Node operator+(Node a,Node b){
16     int i=0,j=0;
17     Node aux;
18     while (i< sz(a.val) && j<sz(b.val)){
19         if (a.val[i]< b.val[j] ) aux.val.pb(a.val[i++]);
20         else aux.val.pb(b.val[j++]);
21     }
22     while (i<sz(a.val)) aux.val.pb(a.val[i++]);
23     while (j<sz(b.val)) aux.val.pb(b.val[j++]);
24     return aux;
25 }
26 //Call segment Tree

```

2.3 Segment Tree Recursive

```

1 struct Node{
2     ll val = 0;
3 };
4 Node operator+(Node a,Node b){return {a.val+b.val};}
5 class Segment_Tree{
6 public:
7     int n;
8     vector<Node> tree;
9
10    Segment_Tree(int x = 1e5+10){
11        n = x;
12        tree.resize(4*n+1);
13    }
14
15    void update(int v, int tl, int tr, int pos, int new_val){
16        if(tl == tr){
17            tree[v] = {new_val};
18            return;
19        }
20
21        int tm = tl+(tr-tl)/2;
22
23        if (pos<=tm)update(v<<1,tl,tm,pos,new_val);
24        else update((v<<1)+1,tm+1,tr,pos,new_val);
25
26        tree[v] = tree[v<<1] + tree[(v<<1)+1];
27    }
28
29    Node query(int v, int tl, int tr, int l, int r){
30        if (l > r)return {0};
31        if (l==tl && r==tr)return tree[v];

```

```

32
33        int tm = tl+(tr-tl)/2;
34        return query(v<<1,tl,tm,l,min(r,tm))+query((v<<1)+1,tm+1,tr,
35            max(l,tm+1),r);
36    };

```

2.4 Lazy

```

1 struct Segment_Tree{
2
3     int n;
4     vector<Node> tree;
5     vector<Lazy> lazy;
6
7     Segment_Tree(int x = 1e5+10){
8         n = x;
9         tree.resize(4*n);
10        lazy.resize(4*n);
11    }
12
13    void push_set(int v,int l,int r){
14        if(!lazy[v].has_set)return;
15        tree[v].val = lazy[v].val*(r-l+1);
16
17        if(l!=r){
18            lazy[v<<1] = lazy[v];
19            lazy[(v<<1)|1] = lazy[v];
20        }
21        lazy[v].has_set = 0;
22        lazy[v].val = 0;
23    }
24
25    void push_sum(int v,int l,int r){
26        if(!lazy[v].has_sum)return;
27        tree[v].val = tree[v].val+lazy[v].val*(r-l+1);
28        if(l!=r){
29            lazy[v<<1].val = lazy[v<<1].val + lazy[v].val;
30            lazy[(v<<1)|1].val = lazy[(v<<1)|1].val + lazy[v].val;
31            if(!lazy[v<<1].has_set)lazy[v<<1].has_sum=1;
32            if(!lazy[(v<<1)|1].has_set)lazy[(v<<1)|1].has_sum=1;
33        }
34        lazy[v].has_sum = 0;
35        lazy[v].val= 0;
36    }
37
38    void update(int pos,ll val,int v,int tl, int tr){
39        if(tl == tr)tree[v] = {val};
40        else{
41            int tm = tl+(tr-tl)/2;
42            if(pos <= tm)update(pos,val,v<<1,tl,tm);
43            else update(pos,val,(v<<1)+1,tm+1,tr);
44            tree[v] = tree[v<<1]+tree[(v<<1)+1];
45        }
46    }
47
48    void update_set(int v,int tl,int tr,int l,int r,ll val){
49        push_set(v,tl,tr);
50        push_sum(v,tl,tr);
51        if(l>tr||r<tl)return;
52        if(tl>= l && tr <=r){
53            lazy[v].has_set=1;
54            lazy[v].has_sum=0;
55            lazy[v].val=val;
56            push_set(v,tl,tr);
57            return;
58        }
59        int tm = tl+(tr-tl)/2;

```

```

60     update_set(v<<1,tl,tm,l,r,val);
61     update_set((v<<1)|1,tm+1,tr,l,r,val);
62     tree[v] = tree[v<<1]+tree[(v<<1)|1];
63 }
64
65 void update_sum(int v,int tl,int tr,int l,int r,ll val){
66     push_set(v,tl,tr);
67     push_sum(v,tl,tr);
68     if(l>tr||r<tl)return;
69     if(tl>= l && tr <=r){
70         lazy[v].has_sum=1;
71         lazy[v].val += val;
72         push_sum(v,tl,tr);
73         return;
74     }
75     int tm = tl+(tr-tl)/2;
76     update_sum(v<<1,tl,tm,l,r,val);
77     update_sum((v<<1)|1,tm+1,tr,l,r,val);
78     tree[v] = tree[v<<1]+tree[(v<<1)|1];
79 }
80
81 Node query(int v ,int tl ,int tr ,int l ,int r){
82     push_set(v,tl,tr);
83     push_sum(v,tl,tr);
84     if(l>tr||r<tl)return {0};
85     if(tl>= l && tr <= r )
86         return tree[v];
87     int tm = tl+(tr-tl)/2;
88     return query(v<<1,tl,tm,l,r)+query((v<<1)|1,tm+1,tr,l,r);
89 }
90
91 };

```

2.5 Persistent Segment Tree

```

1 struct Vertex {
2     Vertex *l, *r;
3     int sum;
4     Vertex(int val) : l(nullptr), r(nullptr), sum(val) {}
5     Vertex(Vertex *l, Vertex *r) : l(l), r(r), sum(0) {}
6     if (l) sum += l->sum;
7     if (r) sum += r->sum;
8 }
9 };
10 Vertex* build(int a[], int tl, int tr) {
11     if (tl == tr)
12         return new Vertex(a[tl]);
13     int tm = (tl + tr) / 2;
14     return new Vertex(build(a, tl, tm), build(a, tm+1, tr));
15 }
16 int get_sum(Vertex* v, int tl, int tr, int l, int r) {
17     if (l > r)
18         return 0;
19     if (l == tl && tr == r)
20         return v->sum;
21     int tm = (tl + tr) / 2;
22     return get_sum(v->l, tl, tm, l, min(r, tm))
23     + get_sum(v->r, tm+1, tr, max(l, tm+1), r);
24 }
25 Vertex* update(Vertex* v, int tl, int tr, int pos, int new_val) {
26     if (tl == tr)
27         return new Vertex(new_val);
28     int tm = (tl + tr) / 2;
29     if (pos <= tm)
30         return new Vertex(update(v->l, tl, tm, pos, new_val), v->r);
31     else
32         return new Vertex(v->l, update(v->r, tm+1, tr, pos, new_val));
33 }

```

2.6 Dynamic Segment Tree

```

1 struct Vertex {
2     int left, right;
3     int sum = 0;
4     Vertex *left_child = nullptr, *right_child = nullptr;
5     Vertex(int lb, int rb) {
6         left = lb;
7         right = rb;
8     }
9     void extend() {
10        if (!left_child && left + 1 < right) {
11            int t = (left + right) / 2;
12            left_child = new Vertex(left, t);
13            right_child = new Vertex(t, right);
14        }
15    }
16    void add(int k, int x) {
17        extend();
18        sum += x;
19        if (left_child) {
20            if (k < left_child->right)
21                left_child->add(k, x);
22            else
23                right_child->add(k, x);
24        }
25    }
26    int get_sum(int lq, int rq) {
27        if (lq <= left && right <= rq)
28            return sum;
29        if (max(left, lq) >= min(right, rq))
30            return 0;
31        extend();
32        return left_child->get_sum(lq, rq) + right_child->get_sum(lq, rq);
33    }
34 };

```

2.7 Sparse Table

```

1 struct Node{
2     int val = 0;
3     Node operator+(Node b){return {__gcd(val,b.val)};}
4 };
5
6 class SparseTable{
7     public:
8     vector<vector<Node>> sparse;
9
10    SparseTable(int n=2e5+10){
11        sparse.assign(n , vector<Node> (log2(n)+1));
12    }
13
14    void build(const vi &a){
15        for(int i=0;i<sz(a);i++)
16            sparse[i][0] = {a[i]};
17
18        for(int j=1;(1<<j)<=sz(a);j++)
19            for(int i=0;i+(1<<j)-1 <sz(a);i++)
20                sparse[i][j]= sparse[i][j-1]+sparse[i+(1<<(j-1))][j-1];
21    }
22
23    int query(int l, int r){
24        int len = r-l+1;
25        int k = log2(len);
26        return (sparse[l][k]+sparse[r-(1<<k)+1][k]).val;
27    }
28
29 };

```

2.8 Treap

```

1 struct node {
2     int val, sz, prior, lazy, sum, mx, mn, repl;
3     bool repl_flag, rev;
4     node *l, *r, *par;
5     node() {
6         lazy = 0;
7         rev = 0;
8         sum = 0;
9         val = 0;
10        sz = 0;
11        mx = 0;
12        mn = 0;
13        repl = 0;
14        repl_flag = 0;
15        prior = 0;
16        l = NULL;
17        r = NULL;
18        par = NULL;
19    }
20    node(int _val) {
21        val = _val;
22        sum = _val;
23        mx = _val;
24        mn = _val;
25        repl = 0;
26        repl_flag = 0;
27        rev = 0;
28        lazy = 0;
29        sz = 1;
30        prior = rnd();
31        l = NULL;
32        r = NULL;
33        par = NULL;
34    }
35 };
36
37 struct treap {
38     typedef node* pnode;
39     pnode root;
40     map<int, pnode> position; //positions of all the values
41     //clearing the treap
42     void clear() {
43         root = NULL;
44         position.clear();
45     }
46
47     treap() {
48         clear();
49     }
50
51     int size(pnode t) {
52         return t ? t->sz : 0;
53     }
54     void update_size(pnode &t) {
55         if(t) t->sz = size(t->l) + size(t->r) + 1;
56     }
57
58     void update_parent(pnode &t) {
59         if(!t) return;
60         if(t->l) t->l->par = t;
61         if(t->r) t->r->par = t;
62     }
63     //add operation
64     void lazy_sum_upd(pnode &t) {
65         if( !t or !t->lazy ) return;

```

```

66     t->sum += t->lazy * size(t);
67     t->val += t->lazy;
68     t->mx += t->lazy;
69     t->mn += t->lazy;
70     if( t->l ) t->l->lazy += t->lazy;
71     if( t->r ) t->r->lazy += t->lazy;
72     t->lazy = 0;
73 }
74 //replace update
75 void lazy_repl_upd(pnode &t) {
76     if( !t or !t->repl_flag ) return;
77     t->val = t->mx = t->mn = t->repl;
78     t->sum = t->val * size(t);
79     if( t->l ) {
80         t->l->repl = t->repl;
81         t->l->repl_flag = true;
82     }
83     if( t->r ) {
84         t->r->repl = t->repl;
85         t->r->repl_flag = true;
86     }
87     t->repl_flag = false;
88     t->repl = 0;
89 }
90 //reverse update
91 void lazy_rev_upd(pnode &t) {
92     if( !t or !t->rev ) return;
93     t->rev = false;
94     swap(t->l, t->r);
95     if( t->l ) t->l->rev ^= true;
96     if( t->r ) t->r->rev ^= true;
97 }
98 //reset the value of current node assuming it now
99 //represents a single element of the array
100 void reset(pnode &t) {
101     if(!t) return;
102     t->sum = t->val;
103     t->mx = t->val;
104     t->mn = t->val;
105 }
106 //combine node l and r to form t by updating corresponding
    queries
107 void combine(pnode &t, pnode l, pnode r) {
108     if(!l) {
109         t = r;
110         return;
111     }
112     if(!r) {
113         t = l;
114         return;
115     }
116     //Beware!!!Here t can be equal to l or r anytime
117     //i.e. t and (l or r) is representing same node
118     //so operation is needed to be done carefully
119     //e.g. if t and r are same then after t->sum=l->sum+r->sum
        operation,
120     //r->sum will be the same as t->sum
121     //so BE CAREFUL
122     t->sum = l->sum + r->sum;
123     t->mx = max(l->mx, r->mx);
124     t->mn = min(l->mn, r->mn);
125 }
126 //perform all operations
127 void operation(pnode &t) {
128     if( !t ) return;
129     reset(t);
130     lazy_rev_upd(t->l);
131     lazy_rev_upd(t->r);
132     lazy_repl_upd(t->l);
133     lazy_repl_upd(t->r);
134     lazy_sum_upd(t->l);
135     lazy_sum_upd(t->r);
136     combine(t, t->l, t);
137     combine(t, t, t->r);
138 }
139 //split node t in l and r by key k
140 //so first k+1 elements(0,1,2,...k) of the array from node t
141 //will be split in left node and rest will be in right node
142 void split(pnode t, pnode &l, pnode &r, int k, int add = 0) {
143     if(t == null) {
144         l = null;
145         r = null;
146         return;
147     }
148     lazy_rev_upd(t);
149     lazy_repl_upd(t);
150     lazy_sum_upd(t);
151     int idx = add + size(t->l);
152     if(t->l) t->l->par = null;
153     if(t->r) t->r->par = null;
154     if(idx <= k)
155         split(t->r, t->r, r, k, idx + 1), l = t;
156     else
157         split(t->l, l, t->l, k, add), r = t;
158
159     update_parent(t);
160     update_size(t);
161     operation(t);
162 }
163 //merge node l with r in t
164 void merge(pnode &t, pnode l, pnode r) {
165     lazy_rev_upd(l);
166     lazy_rev_upd(r);
167     lazy_repl_upd(l);
168     lazy_repl_upd(r);
169     lazy_sum_upd(l);
170     lazy_sum_upd(r);
171     if(!l) {
172         t = r;
173         return;
174     }
175     if(!r) {
176         t = l;
177         return;
178     }
179
180     if(l->prior > r->prior)
181         merge(l->r, l->r, r), t = l;
182     else
183         merge(r->l, l, r->l), t = r;
184
185     update_parent(t);
186     update_size(t);
187     operation(t);
188 }
189 //insert val in position a[pos]
190 //so all previous values from pos to last will be right shifted
191 void insert(int pos, int val) {
192     if(root == NULL) {
193         pnode to_add = new node(val);
194         root = to_add;
195         position[val] = root;
196         return;
197     }
198
199     pnode l, r, mid;
200     mid = new node(val);
201     position[val] = mid;
202
203     split(root, l, r, pos - 1);
204     merge(l, l, mid);
205     merge(root, l, r);
206
207     /*pnode x = new Node(val),merge(root,root,x);*/
208 }
209 //erase from qL to qR indexes
210 //so all previous indexes from qR+1 to last will be left shifted
    qR-qL+1 times
211 void erase(int qL, int qR) {
212     pnode l, r, mid;
213
214     split(root, l, r, qL - 1);
215     split(r, mid, r, qR - qL);
216     merge(root, l, r);
217 }
218 //returns answer for corresponding types of query
219 int query(int qL, int qR) {
220     pnode l, r, mid;
221
222     split(root, l, r, qL - 1);
223     split(r, mid, r, qR - qL);
224
225     int answer = mid->sum;
226     merge(r, mid, r);
227     merge(root, l, r);
228
229     return answer;
230 }
231 //add val in all the values from a[qL] to a[qR] positions
232 void update(int qL, int qR, int val) {
233     pnode l, r, mid;
234
235     split(root, l, r, qL - 1);
236     split(r, mid, r, qR - qL);
237     lazy_repl_upd(mid);
238     mid->lazy += val;
239
240     merge(r, mid, r);
241     merge(root, l, r);
242 }
243 //reverse all the values from qL to qR
244 void reverse(int qL, int qR) {
245     pnode l, r, mid;
246
247     split(root, l, r, qL - 1);
248     split(r, mid, r, qR - qL);
249
250     if(mid)mid->rev ^= 1;
251     merge(r, mid, r);
252     merge(root, l, r);
253 }
254 //replace all the values from a[qL] to a[qR] by v
255 void replace(int qL, int qR, int v) {
256     pnode l, r, mid;
257
258     split(root, l, r, qL - 1);
259     split(r, mid, r, qR - qL);
260     lazy_sum_upd(mid);
261     mid->repl_flag = 1;
262     mid->repl = v;
263     merge(r, mid, r);

```

```

264     merge(root, l, r);
265 }
266 //it will cyclic right shift the array k times
267 //so for k=1, a[qL]=a[qR] and all positions from qL+1 to qR will
268 //have values from previous a[qL] to a[qR-1]
269 //if you make left_shift=1 then it will to the opposite
270 void cyclic_shift(int qL, int qR, int k, bool left_shift = 0) {
271     if(qL == qR) return;
272     k %= (qR - qL + 1);
273
274     pnode l, r, mid, fh, sh;
275     split(root, l, r, qL - 1);
276     split(r, mid, r, qR - qL);
277
278     if(left_shift == 0) split(mid, fh, sh, (qR - qL + 1) - k - 1);
279     else split(mid, fh, sh, k - 1);
280     merge(mid, sh, fh);
281
282     merge(r, mid, r);
283     merge(root, l, r);
284 }
285 bool exist;
286 //returns index of node curr
287 int get_pos(pnode curr, pnode son = nullptr) {
288     if(exist == 0) return 0;
289     if(curr == NULL) {
290         exist = 0;
291         return 0;
292     }
293     if(!son) {
294         if(curr == root) return size(curr->l);
295         else return size(curr->l) + get_pos(curr->par, curr);
296     }
297
298     if(curr == root) {
299         if(son == curr->l) return 0;
300         else return size(curr->l) + 1;
301     }
302
303     if(curr->l == son) return get_pos(curr->par, curr);
304     else return get_pos(curr->par, curr) + size(curr->l) + 1;
305 }
306 //returns index of the value
307 //if the value has multiple positions then it will
308 //return the last index where it was added last time
309 //returns -1 if it doesn't exist in the array
310 int get_pos(int value) {
311     if(position.find(value) == position.end()) return -1;
312     exist = 1;
313     int x = get_pos(position[value]);
314     if(exist == 0) return -1;
315     else return x;
316 }
317 //returns value of index pos
318 int get_val(int pos) {
319     return query(pos, pos);
320 }
321 //returns size of the treap
322 int size() {
323     return size(root);
324 }
325 //inorder traversal to get indexes chronologically
326 void inorder(pnode cur) {
327     if(cur == NULL) return;
328     operation(cur);
329     inorder(cur->l);
330     cout << cur->val << ' ';

```

```

331     inorder(cur->r);
332 }
333
334 bool find(int val) {
335     if(get_pos(val) == -1) return 0;
336     else return 1;
337 }
338 };
339
340 treap t;
341 //Beware!!!here treap is 0-indexed
342
343 int main() {
344     int i, j, k, n, m, l, r, q;
345     for(i = 0; i < 10; i++) t.insert(i, i * 10);
346     t.cyclic_shift(4, 5, 1);
347     t.update(2, 5, 1);
348     t.replace(2, 5, 100);
349     t.reverse(2, 9);
350     t.replace(2, 5, 200);
351     cout << t.query(0, 7) << nl;
352     t.cyclic_shift(2, 3, 2, 1);
353     cout << t.get_pos(20) << nl;
354     t.erase(2, 2);
355     cout << t.find(30) << nl;
356     t.print_array();
357     return 0;
358 }

```

2.9 BIT

```

1 struct BIT{
2     vll bit;
3     int n;
4     BIT(int x){n = x+1;bit.assign(n);}
5
6     ll get(int x){
7         ll ans = 0;
8         for(;x; x-=x&-x)ans+=bit[x];
9         return ans;
10    }
11
12    void update(int x, ll val){
13        for(;x<n; x+=x&-x)
14            bit[x] += val;
15    }
16 };

```

2.10 DSU

```

1 class DSU{
2     private:
3     vi parent,rank,size;
4     int c;
5     public:
6     int mx = 0;
7     DSU(int n):parent(n+1),rank(n+1,0),size(n+1,1),c(n){
8         iota(all(parent),0);
9     }
10
11    int find(int u){return parent[u] == u?u:parent[u] = find(parent[
12        u]);}
13
14    bool same(int u,int v){return find(u)==find(v);}
15
16    int get_size(int u){return size[find(u)];}
17
18    int count(){return c;}

```

```

18
19 void merge(int u,int v){
20     u = find(u);
21     v = find(v);
22     if(u!=v){
23         c--;
24         if(rank[u] > rank[v])swap(u,v);
25         parent[u] = v;
26         size[v] += size[u];
27         if(rank[u] == rank[v])rank[v]++;
28         mx = max(mx,size[v]);
29     }
30 }
31 };

```

2.11 Trie

```

1 struct Node{
2     int v;
3     int cnt = 0;
4     Node* nxt[2];
5     Node(int __v){
6         v = __v;
7         cnt = 0;
8         nxt[0] = nxt[1] = NULL;
9     }
10 };
11
12 struct Trie{
13     typedef Node* pnode;
14     pnode root;
15     const int B = 30;
16
17     void insert(int val){
18         pnode cur = root;
19         for(int i=B;i>=0;i--){
20             bool b = val>>i&1;
21             if(cur->nxt[b]==NULL)cur->nxt[b] = new Node(b);
22             cur = cur->nxt[b];
23             cur->cnt++;
24         }
25     }
26
27     pnode erase(pnode cur, int val, int i) {
28         if (cur == NULL)return NULL;
29         cur->cnt--;
30         if (i < 0) {
31             if (cur->cnt == 0) {
32                 delete cur;
33                 return NULL;
34             }
35             return cur;
36         }
37
38         bool b = (val >> i) & 1;
39         cur->nxt[b] = erase(cur->nxt[b], val, i - 1);
40
41         if (cur->cnt == 0) {
42             delete cur;
43             return NULL;
44         }
45
46         return cur;
47     }
48
49     int query(int val){
50         pnode cur = root;
51         int ans = 0;

```

```

52     for(int i = B;i>=0;i--){
53         bool b = val>>i&1;
54         if(cur->nxt[!b])cur = cur->nxt[!b],ans<=<1,ans++;
55         else cur = cur->nxt[b],ans<=<1;
56     }
57     return ans;
58 }
59 };

```

3 Graphs

3.1 2-SAT

```

1  class SAT{
2  private:
3      int n;
4      vector<vi> adj;
5      vector<vi> adj_t;
6      vector<vi> adj_cond;
7      vi order;
8      vector<bool> vis;
9      vi who;
10     vector<bool> ans;
11     int comp;
12 public:
13
14     SAT(int m=0){
15         n = (m<<1);
16         adj.resize(n);
17         adj_t.resize(n);
18         vis.assign(n,0);
19         who.assign(n,-1);
20     }
21
22     void dfs1(int u) {
23         vis[u] = true;
24         for (int v:adj[u])
25             if (!vis[v])dfs1(v);
26         order.pb(u);
27     }
28
29     void dfs2(int u) {
30         vis[u] = 1;
31         for (int v : adj_t[u])
32             if (!vis[v])dfs2(v);
33         who[u] = comp;
34     }
35
36     //~a->b ~b->a
37     void addOR(int a, bool af, int b, bool bf) {
38         a += a + (af ^ 1);
39         b += b + (bf ^ 1);
40         adj[a ^ 1].push_back(b);
41         adj[b ^ 1].push_back(a);
42         adj_t[b].push_back(a ^ 1);
43         adj_t[a].push_back(b ^ 1);
44     }
45
46     //( a OR b ) ^ (~a or ~b)
47     void addXOR(int a, bool af, int b, bool bf) {
48         addOR(a, af, b, bf);
49         addOR(a, !af, b, !bf);
50     }
51     //(a -> b)
52     void _add(int a,bool af,int b,bool bf) {
53         a += a + (af ^ 1);
54         b += b + (bf ^ 1);
55         adj[a].push_back(b);

```

```

56     adj_t[b].push_back(a);
57 }
58
59 //(a->b)^(b->a)
60 void add(int a,bool af,int b,bool bf) {
61     _add(a, af, b, bf);
62     _add(b, !bf, a, !af);
63 }
64
65 bool solve_SAT(){
66     for(int i=0;i<n;i++)
67         if(!vis[i])dfs1(i);
68
69     comp = -1;
70     vis.assign(n,0);
71     for(int i=sz(order)-1;i>=0;i--){
72         if(!vis[order[i]]){
73             comp++;
74             dfs2(order[i]);
75         }
76         ans.assign(n/2,0);
77         for(int i=0;i<n;i+=2){
78             if(who[i] == who[i+1])return 0;
79             ans[i/2] = (who[i] > who[i+1]);
80         }
81         return 1;
82     }
83
84     bool operator[](int i){
85         return ans[i];
86     }
87
88 };

```

3.2 Block Cut Tree

```

1  vector<vi> adj(n+1),bcc(n+1);
2  vi disc(n+1),low(n+1),who(n+1);
3  stack<int> st;
4  int Time = 0,sz = 0,x;
5
6  auto dfs = [&](auto &self,int u,int p)->void{
7      disc[u] = low[u] = ++Time;
8      st.push(u);
9      for(int v:adj[u]){
10         if(!disc[v]){
11             self(self,v,u);
12             low[u] = min(low[u],low[v]);
13             if(low[v] >= disc[u]){
14                 sz++;
15                 do{
16                     x = st.top();
17                     st.pop();
18                     bcc[x].pb(sz);
19                 }while(x^v);
20                 bcc[u].pb(sz);
21             }
22             }else if(v!=p)low[u] = min(low[u],disc[v]);
23         }
24     };
25
26     for(int i=1;i<=n;i++){
27         if(!disc[i])dfs(dfs,i,0);
28     }
29     int c = sz+n+2;
30     vector<vi> tree(c);
31     vector<bool> art(c,0);
32     for(int i=1;i<=n;i++){

```

```

33     if(sz(bcc[i])>1){
34         who[i] = ++sz;
35         art[who[i]] = 1;
36         for(int v:bcc[i]){
37             tree[who[i]].pb(v);
38             tree[v].pb(who[i]);
39         }
40     }else if(sz(bcc[i])==1)who[i] = bcc[i][0];
41 }

```

3.3 Bellman-Ford Cycle

```

1  vector<int> d(n, INF);
2  d[v] = 0;
3  vector<int> p(n, -1);
4  int x;
5  for (int i = 0; i < n; ++i) {
6      x = -1;
7      for (Edge e : edges)
8          if (d[e.a] < INF)
9              if (d[e.b] > d[e.a] + e.cost) {
10                 d[e.b] = max(-INF, d[e.a] + e.cost);
11                 p[e.b] = e.a;
12                 x = e.b;
13             }
14     }
15     if (x == -1)cout << "No negative cycle from " << v;
16     else{
17         int y = x;
18         for (int i = 0; i < n; ++i)y = p[y];
19         vector<int> path;
20         for (int cur = y;; cur = p[cur]) {
21             path.push_back(cur);
22             if (cur == y && path.size() > 1)break;
23         }
24         reverse(path.begin(), path.end());
25     }

```

3.4 Bellman-Ford Path

```

1  vector<int> d(n, INF);
2  d[v] = 0;
3  vector<int> p(n, -1);
4
5  for (;;) {
6      bool any = false;
7      for (Edge e : edges)
8          if (d[e.a] < INF)
9              if (d[e.b] > d[e.a] + e.cost) {
10                 d[e.b] = d[e.a] + e.cost;
11                 p[e.b] = e.a;
12                 any = true;
13             }
14     if (!any)
15         break;
16 }
17
18 if (d[t] == INF)
19     cout << "No path from " << v << " to " << t << ".";
20 else {
21     vector<int> path;
22     for (int cur = t; cur != -1; cur = p[cur])path.push_back(cur);
23     reverse(path.begin(), path.end());
24 }

```

3.5 BFS

```

1  queue<int> q;
2  q.push(0);

```

```

3 while(!q.empty()){
4     int u = q.front();
5     q.pop();
6     for(int v:adj[u]){
7         if (!vis[v]) {
8             vis[v] = 1;
9             q.push(v);
10        }
11    }
12 }

```

3.6 Bipartite Check

```

1 vector<vi> color;
2 bool ok = 1;
3 queue<int> q;
4 for(int i=0;i<n;i++){
5     if(a[i]!=0)continue;
6     q.push(i);
7     color[i] = 1;
8     while (!q.empty()) {
9         int v = q.front();
10        q.pop();
11        for(int u:adj[v]){
12            if(a[u]==0){
13                a[u] = a[v]^3;
14                q.push(u);
15            }else{
16                valid &= a[u]!=a[v];
17            }
18        }
19    }
20 }

```

3.7 Bridges

```

1
2 vector<pii> bridge;
3 vi disc(MAX),low(MAX);
4 vector<vi> adj(MAX);
5 int Time = 0;
6
7 void dfs(int u,int p){
8     disc[u] = low[u] = ++Time;
9     for(int v:adj[u]){
10        if(v==p)continue;
11        if(!disc[v]){
12            dfs(v,u);
13            // <= for articulation points
14            if(disc[u]<low[v])bridge.emplace_back(u,v);
15            low[u] = min(low[u],low[v]);
16        }else{
17            low[u] = min(low[u],disc[v]);
18        }
19    }
20 }
21 }

```

3.8 Bridge Tree

```

1 //Bridges already found
2 vi who(n+1,-1);
3 int c = 1;
4
5 auto dfs2 = [&](auto &self,int u)->void{
6     who[u] = c;
7     for(auto [v,i]:adj[u]){
8         if(who[v]!=-1||bridge[i])continue;
9         self(self,v);

```

```

10    }
11 };
12
13
14 for(int i=1;i<=n;i++){
15     if(who[i]==-1)dfs2(dfs2,i),c++;
16 }
17
18 vector<vi> tree(c);
19
20 for(int i=0;i<m;i++){
21     if(bridge[i]){
22         tree[who[edges[i].u]].pb(who[edges[i].v]);
23         tree[who[edges[i].v]].pb(who[edges[i].u]);
24     }
25 }

```

3.9 DFS

```

1 void dfs(int u){
2     vis[u] = 1;
3     for(auto v:adj[u]){
4         if(!vis[v])dfs(v);
5     }
6 }

```

3.10 Dijkstra

```

1 vll dist(n,INF64);
2 vector<bool> vis(n,0);
3 priority_queue<pll,vector<pll>,greater<pll>> pq;
4 pq.emplace(0,0);
5 dist[0] = 0;
6 vll cnt(n,0);//number of shortest paths
7 while(!pq.empty()){
8     auto [di,u] = pq.top();
9     pq.pop();
10    if(vis[u])continue;
11    vis[u] = 1;
12    for(auto [v,c]:adj[u]){
13        if(dist[u]+c < dist[v]){
14            dist[v] = dist[u]+c;
15            pq.emplace(dist[v],v);
16            cnt[v] = cnt[u];
17        }else if(dist[u]+c == dist[v]){cnt[v]+=cnt[u]}%MOD;
18    }
19 }

```

3.11 Dinic

```

1 struct flowEdge{
2     int u,v;
3     ll cap,flow = 0;
4     flowEdge(int u, int v, ll cap) : u(u), v(v), cap(cap) {};
5 };
6
7 struct Dinic{
8     vector<flowEdge> edges;
9     vector<vi> adj;
10    int n,s,t;
11    int id = 0;
12    vi level, next;
13
14    queue<int> q;
15
16    Dinic(int n, int s, int t) : n(n), s(s), t(t) {
17        adj.resize(n);
18        level.resize(n);
19        next.resize(n);

```

```

20        fill(all(level),-1);
21        level[s] = 0;
22        q.push(s);
23    }
24
25    void add(int u, int v, ll cap){
26        edges.emplace_back(u,v,cap);
27        edges.emplace_back(v,u,0);
28        adj[u].pb(id++);
29        adj[v].pb(id++);
30    }
31
32    bool bfs(){
33        while (!q.empty()){
34            int curr = q.front();
35            q.pop();
36            for (auto e : adj[curr]){
37                if (edges[e].cap - edges[e].flow < 1) continue;
38                if (level[edges[e].v] != -1) continue;
39                level[edges[e].v] = level[edges[e].u] + 1;
40                q.push(edges[e].v);
41            }
42        }
43        return level[t] != -1;
44    }
45
46    ll dfs(int u, ll flow){
47        if (flow == 0) return 0;
48        if (u == t) return flow;
49        for (auto& cid = next[u]; cid < adj[u].size(); cid++){
50            int e = adj[u][cid];
51            int v = edges[e].v;
52
53            if (level[edges[e].u] + 1 != level[v] || edges[e].cap -
edges[e].flow < 1) continue;
54            ll f = dfs(v,min(flow,edges[e].cap - edges[e].flow));
55            if (f == 0) continue;
56            edges[e].flow += f;
57            edges[e ^ 1].flow -= f;
58            return f;
59        }
60        return 0;
61    }
62
63    ll maxFlow(){
64        ll flow = 0;
65        while (bfs()){
66            fill(all(next),0);
67            for (ll f = dfs(s,INF64); f != 0ll; f = dfs(s,INF64)) flow
+= f;
68            fill(all(level),-1);
69            level[s] = 0;
70            q.push(s);
71        }
72        return flow;
73    }
74
75
76    vector<pii> minCut(){
77        vector<pii> ans;
78        fill(all(level),-1);
79        level[s] = 0;
80        q.push(s);
81        while (!q.empty()){
82            int curr = q.front();
83            q.pop();
84            for (int id = 0; id < adj[curr].size(); id++){

```



```

85     int e = adj[curr][id];
86     if (level[edges[e].v] == -1 && edges[e].cap - edges[e].flow
    > 0){
87         q.push(edges[e].v);
88         level[edges[e].v] = level[edges[e].u] + 1;
89     }
90 }
91 }
92
93 for (int i = 0; i < level.size(); i++){
94     if (level[i] != -1){
95         for (int id = 0; id < adj[i].size(); id++){
96             int e = adj[i][id];
97             if (level[edges[e].v] == -1 && edges[e].cap - edges[e].flow
    == 0)
98                 ans.emplace_back(edges[e].u, edges[e].v);
99         }
100     }
101 }
102 return ans;
103 }
104
105 vector<pii> MaximumMatching(){
106     vector<pii> ans;
107     fill(all(level), -1);
108     level[s] = 0;
109     q.push(s);
110     while (!q.empty()){
111         int curr = q.front();
112         q.pop();
113         for (int id = 0; id < adj[curr].size(); id++){
114             int e = adj[curr][id];
115             if (level[edges[e].v] == -1 && edges[e].cap - edges[e].flow ==
    0 && edges[e].flow != 011){
116                 q.push(edges[e].v);
117                 level[edges[e].v] = level[edges[e].u] + 1;
118             }
119         }
120     }
121     for (int i = 0; i < level.size(); i++){
122         if (level[i] != -1)
123             for (int id = 0; id < adj[i].size(); id++){
124                 int e = adj[i][id];
125                 if (edges[e].u != s && edges[e].v != t && edges[e].cap -
    edges[e].flow == 0 && edges[e].flow != 011)
126                     ans.emplace_back(edges[e].u, edges[e].v);
127             }
128     }
129     return ans;
130 }
131
132 };

```

3.12 Euler Path Directed

```

1 struct EulerSolver{
2 private:
3     vector<vector<pii>> adj;
4     int n, edges;
5     vi in, out;
6     vi ans, edge_ans;
7     int root;
8     vi done;
9     void dfs(int u){
10         while(done[u] < sz(adj[u])){
11             auto [v, id] = adj[u][done[u]++];
12             dfs(v);
13             edge_ans.pb(id);

```

```

14         }
15         ans.pb(u);
16     }
17 public:
18     EulerSolver(vector<vector<pii>> &a, int t):adj(a){
19         n = t;
20         edges = 0;
21         in.assign(n, 0);
22         out.assign(n, 0);
23         done.assign(n, 0);
24         root = -1;
25         for(int u=0; u<n; u++){
26             for(auto v:adj[u]){
27                 edges++;
28                 in[v.F]++;
29                 out[u]++;
30             }
31         }
32     }
33
34     bool possible(){
35         int cnt1 = 0, cnt2 = 0;
36         bool ok = 1;
37         for(int i=0; i<n; i++){
38             if (in[i] - out[i] == 1) cnt1++;
39             if (out[i] - in[i] == 1) cnt2++;
40             if (abs(in[i] - out[i]) > 1) ok = 0;
41         }
42         if(cnt1 > 1 || cnt2 > 1) ok = 0;
43         if(!ok) return 0;
44
45         if(root == -1)
46             for(int i=0; i<n; i++){
47                 if(out[i]) root = i;
48             }
49         if(root == -1) return 1;
50         dfs(root);
51
52         if(sz(ans) != edges+1) return 0;
53         reverse(all(ans));
54         reverse(all(edge_ans));
55         return 1;
56     }
57
58     int size(){
59         return ans.size();
60     }
61
62     int operator()(int i, bool x){
63         if(x) return ans[i];
64         else return edge_ans[i];
65     }
66 };

```

3.13 Euler Path Undirected

```

1 struct EulerSolver{
2 private:
3     vector<vector<pii>> adj;
4     int n, edges;
5     vi ans, edge_ans;
6     vi deg;
7     int root;
8     vi done;
9     vector<bool> vis;
10
11     void dfs(int u){
12         while(done[u] < sz(adj[u])){

```

```

13             auto [v, id] = adj[u][done[u]++];
14             if(vis[id]) continue;
15             vis[id] = 1;
16             dfs(v);
17             edge_ans.pb(id);
18         }
19         ans.pb(u);
20     }
21 public:
22     EulerSolver(vector<vector<pii>> &a, int t):adj(a), n(t){
23         edges = 0;
24         done.assign(n, 0);
25         deg.assign(n, 0);
26         root = -1;
27         for(int u=0; u<n; u++){
28             for(auto v:adj[u]){
29                 edges++;
30                 deg[v.F]++;
31                 deg[u]++;
32             }
33             vis.assign(edges/2, 0);
34         }
35
36         bool possible(){
37             int odd = 0;
38             for(int i=0; i<n; i++){
39                 if((deg[i]/2)&1){
40                     odd++;
41                     root = i;
42                 }
43             }
44
45             if(odd>2) return 0;
46
47             if(root == -1)
48                 for(int i=0; i<n; i++){
49                     if(deg[i]){root = i; break;}
50                 }
51             if(root == -1) return 1;
52             dfs(root);
53
54             if(sz(ans) != edges/2+1) return 0;
55             reverse(all(ans));
56             reverse(all(edge_ans));
57             return 1;
58         }
59
60         int size(){
61             return ans.size();
62         }
63
64         int operator()(int i, bool x){
65             if(x) return ans[i];
66             else return edge_ans[i];
67         }
68 };

```

3.14 Flood Fill

```

1 int dx[] = {1, 1, 0, -1, -1, -1, 0, 1};
2 int dy[] = {0, 1, 1, 1, 0, -1, -1, -1};
3
4 int floodfill(int x, int y, char color1, char color2){
5     if(grid[x][y] == "outside") return 0;
6     if(grid[x][y] != color1) return 0;
7     int ans = 1;
8     grid[x][y] = color2;

```



```

9  for(int d=0;d<8;d++)ans += floodfill(x+dx[d],y+dy[d],color1,
    colr2);
10 return ans;
11 }

```

3.15 Floyd-Warshall

```

1  for (int i = 0; i < n; ++i)d[i][i] = 0;
2
3  for (int k = 0; k < n; ++k)
4      for (int i = 0; i < n; ++i)
5          for (int j = 0; j < n; ++j)
6              if (d[i][k] < INF && d[k][j] < INF)
7                  d[i][j] = min(d[i][j], d[i][k] + d[k][j]);

```

3.16 Kruskal

```

1  struct Edge{
2      int u,v,w;
3      bool operator < (Edge b){return w<b.w;}
4  };
5
6  //inside main
7
8  vector<Edge> edges;
9  sort(all(edges));
10
11 DSU dsu(vertices);
12
13 for(int i=0;i<E;i++){
14     Edge front = edges[i];
15     if(!dsu.same(front.u,front.v)){
16         cost+= front.w;
17         dsu.merge(front.u,front.v);
18     }
19 }

```

3.17 MCMF

```

1  struct MCMF{
2      struct edge{
3          int u, v;
4          ll cap, cost;
5          int id;
6          edge(int _u, int _v, ll _cap, ll _cost, int _id) {
7              u = _u;
8              v = _v;
9              cap = _cap;
10             cost = _cost;
11             id = _id;
12         }
13     };
14
15     int n, s, t, mxid;
16     ll flow, cost;
17     vector<vi> g;
18     vector<edge> e;
19     vll d, potential, flow_through;
20     vi par;
21     bool neg;
22
23     MCMF(){}
24     //0 indexed
25     MCMF(int _n){
26         n = _n + 10;
27         g.assign(n, vector<int> ());
28         neg = false;
29         mxid = 0;
30     }

```

```

31
32 void add_edge(int u, int v, ll cap, ll cost, int id = -1, bool
    directed = true) {
33     if(cost < 0) neg = true;
34     g[u].push_back(e.size());
35     e.push_back(edge(u, v, cap, cost, id));
36     g[v].push_back(e.size());
37     e.push_back(edge(v, u, 0, -cost, -1));
38     mxid = max(mxid, id);
39     if(!directed) add_edge(v, u, cap, cost, -1, true);
40 }
41
42 bool dijkstra(){
43     par.assign(n, -1);
44     d.assign(n, INF64);
45     priority_queue <pll,vector<pll>,greater<pll>> q;
46     d[s] = 0;
47     q.push({0, s});
48
49     while (!q.empty()){
50         auto [nw,u] = q.top();
51         q.pop();
52         if(nw != d[u])continue;
53         for (int i = 0; i < sz(g[u]); i++) {
54             int id = g[u][i];
55             int v = e[id].v;
56             ll cap = e[id].cap;
57             ll w = e[id].cost + potential[u] - potential[v];
58             if (d[u]+w < d[v] && cap > 0) {
59                 d[v] = d[u] + w;
60                 par[v] = id;
61                 q.push({d[v], v});
62             }
63         }
64     }
65
66     for (int i = 0; i < n; i++){
67         if (d[i] < INF64)
68             d[i] += (potential[i] - potential[s]);
69     }
70
71     for (int i = 0; i < n; i++){
72         if (d[i] < INF64) potential[i] = d[i];
73     }
74     return d[t] != INF64; // for max flow min cost
75     // return d[t] <= 0; // for min cost flow
76 }
77
78 ll send_flow(int v, ll cur){
79     if(par[v] == -1) return cur;
80     int id = par[v];
81     int u = e[id].u;
82     ll w = e[id].cost;
83     ll f = send_flow(u, min(cur, e[id].cap));
84     cost += f * w;
85     e[id].cap -= f;
86     e[id ^ 1].cap += f;
87     return f;
88 }
89
90 pll solve(int _s, int _t, ll goal = INF64) {
91     s = _s;
92     t = _t;
93     flow = 0, cost = 0;
94     potential.assign(n, 0);
95     if(neg){
96         d.assign(n, INF);

```

```

97     d[s] = 0;
98     bool relax = true;
99     for (int i = 0; i < n && relax; i++) {
100         relax = false;
101         for (int u = 0; u < n; u++) {
102             for (int k = 0; k < (int)g[u].size(); k++) {
103                 int id = g[u][k];
104                 int v = e[id].v;
105                 ll cap = e[id].cap, w = e[id].cost;
106                 if (d[v] > d[u] + w && cap > 0) {
107                     d[v] = d[u] + w;
108                     relax = true;
109                 }
110             }
111         }
112     }
113     for(int i = 0; i < n; i++) if(d[i] < INF) potential[i] = d[i];
114 }
115 while (flow < goal && dijkstra()) flow += send_flow(t, goal -
    flow);
116 flow_through.assign(mxid + 10, 0);
117 for (int u = 0; u < n; u++){
118     for (auto v : g[u])
119         if (e[v].id >= 0) flow_through[e[v].id] = e[v ^ 1].cap;
120 }
121
122 return make_pair(flow, cost);
123 }
124 };

```

3.18 Prims

```

1  vector<bool> vis(n);
2  priority_queue<pii, vector<pii>, greater<pii>> q;
3
4  auto process = [&](int u){
5      vis[u] = 1;
6      for(int i=0;i<sz(adj[u]);i++){
7          pii v = adj[u][i];
8          swap(v.S,v.F);
9          q.push(v);
10     }
11 }
12 process(0);
13
14 while(!q.empty()){
15     auto [w,u] = q.top();q.pop();
16     if(!vis[u]){cost += w;process(u);}
17 }

```

3.19 SCC Kosaraju

```

1  class SCC{
2  private:
3      int n;
4      vector<vi> adj;
5      vector<vi> adj_t;
6      vector<vi> adj_cond;
7      vi order;
8      vector<bool> vis;
9      vi who;
10     int comp;
11 public:
12     SCC(int n,vector<vi> &a,vector<vi>&b):n(n),adj(a),adj_t(b){
13         comp = -1;
14         who.resize(n);
15         vis.resize(n);

```

```

16     for(int i=0;i<n;i++){
17         if(!vis[i])dfs1(i);
18     }
19     vis.assign(n,0);
20     for(int i=n-1;i>=0;i--){
21         if(!vis[order[i]]){
22             comp++;
23             dfs2(order[i]);
24         }
25         adj_cond.resize(comp+1);
26         for(int u=0;u<n;u++){
27             for(auto v:adj[u])
28                 if(who[v]!=who[u])
29                     adj_cond[who[u]].pb(who[v]);
30     }
31
32     void dfs1(int u) {
33         vis[u] = true;
34         for (int v:adj[u])
35             if (!vis[v])dfs1(v);
36         order.pb(u);
37     }
38
39     void dfs2(int u) {
40         vis[u] = 1;
41         for (int v : adj_t[u])
42             if (!vis[v])dfs2(v);
43         who[u] = comp;
44     }
45 };

```

3.20 State Space Graph

```

1 vector<vll> dist(n,vll(1001,INF64));
2 vector<vector<bool>> vis(n,vector<bool> (1001,0));
3 typedef array<ll,3> T;
4 priority_queue<T ,vector<T>,greater<T>> pq;
5 pq.push({0,0,state[0]});
6 dist[0][state[0]] = 0;
7 while(!pq.empty()){
8     auto [d,u,s] = pq.top();
9     pq.pop();
10    if(vis[u][s])continue;
11    vis[u][s] = 1;
12    for(auto [v,w]:adj[u]){
13        ll c = min(s,slow[v]);
14        //Nuevo estad cuando llegue a V
15        if(dist[u][s]+w*s<dist[v][c]){
16            dist[v][c] = dist[u][s]+w*s;
17            pq.push({dist[v][c],v,c});
18        }
19    }
20 }

```

3.21 Topological Sort

```

1 vi ts;
2 queue<int> q;
3 for (int i = 1; i < n+1; i++)
4     if(!in[i])q.push(i);
5
6 while (!q.empty()) {
7     int u=q.front();
8     q.pop();
9     ts.pb(u);
10    for (int v : adj[u]) {
11        in[v]--;
12        if(!in[v])q.push(i);

```

```

13    }
14 }
15
16 if(sz(ts)!=n)cout <<"IMPOSSIBLE";

```

4 Trees

4.1 Centroid Decomposition

```

1 vector<bool> removed(n+1);
2 vi sub(n+1);
3
4 auto dfs = [&](auto &self,int u,int p)->void{
5     sub[u] = 1;
6     for(int v:adj[u]){
7         if(v!=p && !removed[v]){
8             self(self,v,u);
9             sub[u] += sub[v];
10        }
11    }
12 };
13
14 auto centroid = [&](auto &self,int u,int p,int root)->int{
15     for(int v:adj[u]){
16         if(v!=p && !removed[v]){
17             if(sub[v]*2 > sub[root])
18                 return self(self,v,u,root);
19         }
20     }
21     return u;
22 };
23
24 auto decompose = [&](auto &self,int u, int p)->void{
25     dfs(dfs,u,p);
26     int c = centroid(centroid,u,p,u);
27
28     //get info from the paths that contains c
29
30     removed[c] = 1;
31
32     for(int v:adj[c]){
33         if(v!=p && !removed[v])
34             self(self,v,c);
35     }
36 }
37 };

```

4.2 Tree Diameter

```

1 vector<vector<pii>> adj;
2 vector<bool> vis;
3
4 int bfs(int n,vll &d,int v = 0){
5     vis.assign(n,0);
6     d.assign(n,0);
7     d[v] = 0;
8     queue<int> q;
9     q.push(v);
10    pll last = {v,0};
11    vis[v] = 1;
12    while(!q.empty()){
13        int u = q.front();q.pop();
14        for(auto [w,c]:adj[u]){
15            if(vis[w])continue;
16            d[w] = d[u]+c;
17            q.push(w);
18            vis[w] = 1;
19            if(d[w]>last.S)last = {w,d[w]};

```

```

20    }
21 }
22 return int(last.F);
23 }
24
25 void solve(){
26     int n;cin >> n;
27     adj.assign(n,vector<pii> ());
28     for(int i=1;i<n;i++){
29         int a,b;ll c;
30         cin >> a >> b >> c;
31         adj[--a].pb({--b,c});
32         adj[b].pb({a,c});
33     }
34     vll dx(n),dy(n);
35     int a =bfs(n,dx);
36     int b =bfs(n,dy,a);
37     bfs(n,dx,b);
38
39     for(int i=0;i<n;i++){
40         cout << max(dx[i],dy[i]) << " \n"[i==(n-1)];
41     }
42 }
43 }

```

4.3 Euler Tour

```

1 int timer = 0;
2 vi start,final;
3 vector<vi> adj;
4 void dfs(int u = 0,int prev = -1){
5     start[u]=timer++;
6     for(int v:adj[u]){
7         if(v!=prev)dfs(v,u);
8     }
9     final[u] = timer;
10 }
11 // [start,final)

```

4.4 Heavy Light Decomposition

```

1 class HLD{
2 private:
3     int n;
4     vector<vi> adj;
5     vi parent;
6     vi heavy;
7     vi depth;
8     vi root;
9     vi treePos;
10    Segment_Tree tree;
11
12    int dfs(int u =0){
13        int size = 1, mx_sub = 0;
14        for(auto v:adj[u])
15            if(v!=parent[u]){
16                parent[v] = u;
17                depth[v] = depth[u]+1;
18                int sub = dfs(v);
19                if(sub > mx_sub){
20                    heavy[u] = v;
21                    mx_sub = sub;
22                }
23                size += sub;
24            }
25        return size;
26    }
27 }

```

```

28 template <class BinaryOperation>
29 void processPath(int u, int v, BinaryOperation op) {
30     for (; root[u] != root[v]; v = parent[root[v]]) {
31         if (depth[root[u]] > depth[root[v]]) swap(u, v);
32         op(treePos[root[v]], treePos[v] + 1);
33     }
34     if (depth[u] > depth[v]) swap(u, v);
35     op(treePos[u], treePos[v] + 1);
36 }
37
38 void init(vi &a){
39     for(int i=0; crt=0; i<n; ++i){
40         if(parent[i] == -1 || heavy[parent[i]]!=i){
41             for(int j = i; j!=-1; j = heavy[j]){
42                 root[j] = i;
43                 treePos[j] = crt++;
44             }
45         }
46     }
47     tree = Segment_Tree(n);
48     for(int i=0; i<n; i++){
49         tree.update(treePos[i], a[i]);
50     }
51 }
52
53 public:
54
55 HLD(int n, vector<vi> &vale, vi &a): n(n), adj(vale){
56     parent.assign(n, 0);
57     heavy.assign(n, -1);
58     depth.assign(n, 0);
59     root.resize(n);
60     treePos.assign(n, 0);
61     parent[0] = -1;
62     dfs();
63     init(a);
64 }
65
66 void update(int u, int &val){
67     tree.update(treePos[u], val);
68 }
69
70 void updatePath(int u, int v, int value) {
71     processPath(u, v, [this, &value](int l, int r) {
72         tree.update_range(l, r, value);
73     });
74 }
75
76 int query(int u, int v){
77     int ans = 0;
78     processPath(u, v, [this, &ans](int l, int r){
79         ans = max(ans, tree.query(l, r));
80     });
81     return ans;
82 }
83 };

```

4.5 LCA

```

1 const int MAX = 1e5+5, LG=18;
2 vi deep(MAX); //Si tiene peso vi cost(MAX)
3 int par[MAX][LG+1];
4
5 void dfs(int u = 1, int p=0) { // U = raiz del arbol
6     par[u][0] = p;
7     deep[u] = deep[p]+1; //cost[u] += cost[p]
8     for(int i=1; i<=LG; i++) par[u][i] = par[par[u][i-1]][i-1];
9     for(int v: adj[u]){

```

```

10         if(v!=p){
11             dfs(v, u);
12         }
13     }
14 }
15
16 int lca(int u, int v){
17     if(deep[u] < deep[v]) swap(u, v);
18     for(int k=LG; k>=0; k--){
19         if(deep[par[u][k]] >= deep[v])
20             u = par[u][k];
21     if(u==v) return u;
22     for(int k=LG; k>=0; k--){
23         if(par[u][k] != par[v][k])
24             u=par[u][k], v=par[v][k];
25     return par[u][0];
26 }
27
28 int dist(int u, int v){
29     int lc = lca(u, v);
30     return deep[u]+deep[v]-(deep[lc]<<1);
31 }
32
33 int kth(int u, int k){
34     assert(k>=0);
35     for(int i=0; i<=LG; i++){
36         if(k&&(1<<i)) u = par[u][i];
37     return u;
38 }
39
40 int isanc(int u, int g) {
41     int k = dep[u] - dep[g];
42     return k >= 0 && kth(u, k) == g;
43 }

```

4.6 Min/Max Weight on Path

```

1 const int MAX = 1e5+5, LG=18;
2 vector<vector<pii>> adj(MAX, vector<pii> ());
3 vi deep(MAX);
4 int par[MAX][LG+1];
5 int mn[MAX][LG+1];
6 int mx[MAX][LG+1];
7
8 void dfs(int u=1, int p=0, int costmn = INF, int costmx = -INF){
9     par[u][0] = p;
10    deep[u] = deep[p]+1;
11
12    if(p!=0){
13        mn[u][0] = costmn;
14        mx[u][0] = costmx;
15    }
16    for(int i=1; i<=LG; i++){
17        par[u][i] = par[par[u][i-1]][i-1];
18        mn[u][i] = min(mn[u][i-1], mn[par[u][i-1]][i-1]);
19        mx[u][i] = max(mx[u][i-1], mx[par[u][i-1]][i-1]);
20    }
21
22    for(auto [v, t]: adj[u]){
23        if(v!=p){
24            dfs(v, u, t);
25        }
26    }
27 }
28
29 pii minmax(int u, int v){
30     pii ans = {INF, -INF};
31     if(deep[u] < deep[v]) swap(u, v);

```

```

32     for(int k=LG; k>=0; k--){
33         ans.F = min(ans.F, mn[u][k]);
34         ans.S = max(ans.S, mx[u][k]);
35         u = par[u][k];
36     }
37     if(u==v) return ans;
38     for(int k=LG; k>=0; k--){
39         if(par[u][k] != par[v][k]){
40             ans.F = min({ans.F, mn[u][k], mn[v][k]});
41             ans.S = max({ans.S, mx[u][k], mx[v][k]});
42             u=par[u][k], v=par[v][k];
43         }
44     }
45     ans.F = min({ans.F, mn[u][0], mn[v][0]});
46     ans.S = max({ans.S, mx[u][0], mx[v][0]});
47     return ans;
48 }

```

4.7 Intersection Path

```

1 pair<int, int> getCommonPath(int u, int a, int v, int b) {
2     // returns common path between u->a and v->b where u is ancestor a
3     // and v is ancestor b
4     if (!isanc(a, v)) return make_pair(0, 0);
5     int x = lca(a, b);
6     if (dep[v] < dep[u]) {
7         if (isanc(x, u))
8             return make_pair(u, x);
9     }else{
10        if (isanc(x, v))
11            return make_pair(v, x); // v->x is the common path
12    }
13 }
14 pair<int, int> path_union_light(pair<int, int> x, pair<int, int> y)
15 ) {
16     if (x.first == 0) return y; //where {0,0} is a null path
17     if (y.first == 0) return x;
18     vector<int> can = {x.first, x.second, y.first, y.second};
19     int a = can[0];
20     for (int u : can) if (dep[u] > dep[a]) a = u;
21     int b = -1;
22     for (int u : can) if (!isanc(a, u)) {
23         if (b == -1) b = u;
24         if (dep[b] < dep[u]) b = u;
25     }
26     if (b == -1) {
27         b = can[0];
28         for (int u : can) if (dep[u] < dep[b]) b = u;
29         return {a, b};
30     }
31     return {a, b};
32 }
33 // returns the intersection of two paths (a,b) and (c,d)
34 // {0,0} for null path
35 pair<int, int> path_intersection(int a, int b, int c, int d) {
36     if (a == 0 || b == 0) return {0, 0};
37     int u = lca(a, b), v = lca(c, d); // splits both paths in two
38     // chains from lca to one node
39     pair<int, int> x, y;
40     x = getCommonPath(u, a, v, c);
41     y = getCommonPath(u, a, v, d);
42     pair<int, int> a1 = path_union_light(x, y);
43     x = getCommonPath(u, b, v, c);
44     y = getCommonPath(u, b, v, d);
45     pair<int, int> a2 = path_union_light(x, y);
46     return path_union_light(a1, a2);
47 }

```

4.8 Union Path

```

1 pair<int, int> merge(pair<int, int> x, pair<int, int> y) {
2     if (x.first == 0)return y; //where {0,0} is a null path
3     if (y.first == 0)return x;
4     if (x.first == -1 || y.first == -1)    return { -1, -1};
5     vector<int> can = {x.first, x.second, y.first, y.second};
6     int a = can[0];
7     for (int u : can)
8         if (dep[u] > dep[a])
9             a = u;
10    int b = -1;
11    for (int u : can)
12        if (!isanc(a, u)) {
13            if (b == -1) b = u;
14            if (dep[b] < dep[u])    b = u;
15        }
16    if (b == -1) {
17        b = can[0];
18        for (int u : can)
19            if (dep[u] < dep[b])
20                b = u;
21        return {a, b};
22    }
23    int g = lca(a, b);
24    for (int u : can) {
25        if (u == a || u == b)continue;
26        if (dep[u] < dep[g] || (!isanc(a, u) && !isanc(b, u)))return {
27            -1, -1};
28    }
29    return {a, b};
30 }

```

4.9 Prufer Code Build

```

1 auto dfs = [&](auto &self, int u){
2     for (int v:adj[u]) {
3         if (v != parent[u]) {
4             parent[v] = u;
5             dfs(v);
6         }
7     }
8 };
9
10 parent[n] = -1;
11 dfs(dfs, n);
12
13 int ptr = -1;
14 vi degree(n);
15 for(int i=1; i<=n; i++) {
16     degree[i] = sz(adj[i]);
17     if(degree[i]==1 && ptr== -1)
18         ptr = i;
19 }
20
21 vi code(n - 2);
22 int leaf = ptr;
23 for(int i=0; i<n-2; i++){
24     int next = parent[leaf];
25     code[i] = next;
26     if (--degree[next]==1 && next<ptr) {
27         leaf = next;
28     }else{
29         ptr++;
30         while (degree[ptr] != 1)
31             ptr++;
32         leaf = ptr;
33     }

```

34 }

4.10 Prufer Code Restore

```

1 int n = code.size() + 2;
2 vi degree(n, 0);
3 for (int i : code)
4     degree[i]++;
5
6 int ptr = 0;
7 while (degree[ptr] != 1)
8     ptr++;
9
10 int leaf = ptr;
11
12 vector<pii> edges;
13 for(int v:code){
14     edges.emplace_back(leaf, v);
15     if (--degree[v] == 0 && v < ptr) {
16         leaf = v;
17     }else{
18         ptr++;
19         while (degree[ptr] != 1)
20             ptr++;
21         leaf = ptr;
22     }
23 }
24
25 edges.emplace_back(leaf, n-1);

```

4.11 Tree Rerooting

```

1 namespace reroot{
2     const auto exclusive = [] (const auto& a, const auto& base, const
3         auto& merge_into, int vertex){
4         int n = (int)a.size();
5         using Aggregate = decay_t<decltype(base)>;
6         vector<Aggregate> b(n, base);
7         for(int bit=(int)lg(n); bit>=0; --bit){
8             for(int i=n-1; i>=0; --i)b[i]=b[i >> 1];
9             int sz=n-(n<1<bit);
10            for(int i=0; i<sz; ++i){
11                int index = (i >> bit) ^ 1;
12                b[index] = merge_into(b[index], a[i], vertex, i);
13            }
14        }
15        return b;
16    };
17
18    // MergeInto : Aggregate * Value * Vertex(int) * EdgeIndex(int)
19    // -> Aggregate
20    // Base : Vertex(int) -> Aggregate
21    // FinalizeMerge : Aggregate * Vertex(int) * EdgeIndex(int) ->
22    // Value
23
24    const auto rerooter = [] (const auto& g, const auto& base, const
25        auto& merge_into, const auto& finalize_merge){
26        int n = (int)g.size();
27        using Aggregate = decay_t<decltype(base(0))>;
28        using Value = decay_t<decltype(finalize_merge(base(0), 0, 0))
29            >;
30        vector<Value> root_dp(n), dp(n);
31        vector<vector<Value>> edge_dp(n), redge_dp(n);
32
33        vector<int> bfs, parent(n);
34        bfs.reserve(n);
35        bfs.push_back(0);
36        for(int i=0; i<n; ++i){
37            int u = bfs[i];
38            for (auto v : g[u]) {

```

```

32         if (parent[u] == v) continue;
33         parent[v] = u;
34         bfs.push_back(v);
35     }
36 }
37
38 for(int i=n-1; i>=0; --i){
39     int u = bfs[i];
40     int p_edge_index = -1;
41     Aggregate aggregate = base(u);
42     for(int edge_index = 0; edge_index < (int)g[u].size(); ++
43         edge_index){
44         int v = g[u][edge_index];
45         if(parent[u] == v) {
46             p_edge_index = edge_index;
47             continue;
48         }
49         aggregate = merge_into(aggregate, dp[v], u, edge_index);
50     }
51     dp[u] = finalize_merge(aggregate, u, p_edge_index);
52 }
53
54 for (auto u : bfs) {
55     dp[parent[u]] = dp[u];
56     edge_dp[u].reserve(g[u].size());
57     for (auto v : g[u]) edge_dp[u].push_back(dp[v]);
58     auto dp_exclusive = exclusive(edge_dp[u], base(u),
59         merge_into, u);
60     redge_dp[u].reserve(g[u].size());
61     for (int i = 0; i < (int)dp_exclusive.size(); ++i) redge_dp[
62         u].push_back(finalize_merge(dp_exclusive[i], u, i));
63     root_dp[u] = finalize_merge(n > 1 ? merge_into(dp_exclusive
64         [0], edge_dp[u][0], u, 0) : base(u), u, -1);
65     for (int i = 0; i < (int)g[u].size(); ++i) {
66         dp[g[u][i]] = redge_dp[u][i];
67     }
68 }
69
70 //in solve()
71 using Aggregate = int;
72 using Value = int;
73
74 auto base = [&](int vertex) -> Aggregate {
75     //base case
76     return ;
77 };
78
79 auto merge_into = [] (Aggregate vertex_dp, Value neighbor_dp, int
80     vertex, int edge_index) -> Aggregate {
81     //merge childs
82     return vertex_dp;
83 };
84
85 auto finalize_merge = [&](Aggregate vertex_dp, int vertex, int
86     edge_index) -> Value {
87     //return of dfs
88     return vertex_dp;
89 };
90
91 auto [root_dp, edge_dp, redge_dp] = reroot::rerooter(adj, base,
92     merge_into, finalize_merge);

```

4.12 Tree Matching

```
1 vector<vll> dp(2,vll(MAX));
2
3 void dfs(ll u = 0, ll p=-1){
4     dp[0][u] = 0;
5     dp[1][u] = -INF;
6
7     for(ll v:adj[u]){
8         if(v == p)continue;
9         dfs(v,u);
10
11         dp[0][u]+=max(dp[1][v],dp[0][v]);
12
13         ll opt = min(dp[0][v]-dp[1][v],0ll);
14
15         dp[1][u] = max(dp[1][u],opt);
16     }
17
18     dp[1][u] += dp[0][u];
19     dp[1][u]++;
20 }
```

4.13 Rooted Tree Hashing

```
1 auto dfs = [](auto &self,int u,int p,vector<vi> &adj)->pair<ll,ll>
2 >{
3     vector<pair<ll,ll>> shash;
4     for(int v:adj[u]){
5         if(v==p)continue;
6         shash.pb(self(self,v,u,adj));
7     }
8     sort(all(shash));
9
10    ll p1 = P1,p2 = P2;
11    pair<ll,ll> hash = {42,42};
12    for(int i=0;i<sz(shash);i++,(p1==p1)%=MOD1,(p2==p2)%=MOD2){
13        (hash.F += shash[i].F*shash[i].F%MOD1+shash[i].F*p1%MOD1+42)%
14        MOD1;
15        (hash.S += shash[i].S*shash[i].S%MOD2+shash[i].S*p2%MOD2+42)%
16        MOD2;
17    }
18    return hash;
19 };
```

4.14 Tree Isomorphism

```
1 //1-indexed
2
3 auto centers = [&](vector<vi> &adj)->vi{
4     //Diameters of a tree
5     vll du(n+1),dv(n+1);
6     int v =bfs(du,1,adj);
7     int u =bfs(dv,v,adj);
8     bfs(du,u,adj);
9     vi ans;
10    for(int i=1;i<=n;i++)
11        if(du[i]+dv[i]==du[v] && du[i]>=du[v]/2 && dv[i]>=du[v]/2)
12            ans.pb(i);
13    return ans;
14 };
15
16 //in main
17
18 vi a = centers(tree1);
19 vi b = centers(tree2);
20
21 for(auto x:a)
22     for(auto y:b)
```

```
23 //checar isomorfismos si estan rooteados
24 //en x y y
```

5 Number Theory

6 Combinatorics

6.1 nCr

```
1 const int MAX = 2e6;
2 ll factorial[MAX+1];
3 ll inv[MAX+1];
4
5 ll binpow(ll a,ll b){
6     ll ans = 1;
7     a%=MOD;
8     while(b){
9         if(b&1)(ans*=a)%=MOD;
10        (a*=a)%=MOD;
11        b>>=1;
12    }
13    return ans;
14 }
15
16 void precalc(){
17     factorial[0]=1;
18     for (ll i = 1; i <= MAX; i++) {
19         factorial[i] = factorial[i-1]*i%MOD;
20     }
21     inv[MAX] = binpow(factorial[MAX],MOD-2);
22     for (ll i = MAX; i >= 1; i--) {
23         inv[i-1] = inv[i]*i%MOD;
24     }
25 }
26
27 ll ncr(ll a,ll b){
28     return factorial[a]*inv[b]%MOD*inv[a-b]%MOD;
29 }
```

7 Math

7.1 FFT

```
1 typedef complex<double> base;
2
3 void fft(vector<base> &p, bool inv = 0) {
4     int n = p.size(), i = 0;
5     for(int j = 1; j < n - 1; ++j) {
6         for(int k = n >> 1; k > (i ^= k); k >>= 1);
7         if(j < i) swap(p[i], p[j]);
8     }
9
10    for(int l = 1, m; (m = 1 << 1) <= n; l <= m) {
11        double ang = 2 * PI / m *(inv?1.0:-1.0);
12        base wn(cos(ang),sin(ang));
13        for(int i = 0; i < n; i += m) {
14            base w(1,0);
15            for(int j = i, k = i + 1; j < k; ++j, w *= wn) {
16                base t = w * p[j + l];
17                p[j + l] = p[j] - t;
18                p[j] = p[j] + t;
19            }
20        }
21    }
22
23    if(inv) for(int i = 0; i < n; ++i) p[i] /= n;
24 }
25 vector<ll> multiply(vi &a, vi &b) {
```

```
26     int n = a.size(), m = b.size(), t = n + m - 1, sz = 1;
27     while(sz < t) sz <= 1;
28     vector<base> x(sz), y(sz);
29     for(int i = 0; i < n; ++i) x[i] = base(a[i], 0);
30     for(int i = 0; i < m; ++i) y[i] = base(b[i], 0);
31     fft(x), fft(y);
32     for(int i = 0; i < sz; ++i) x[i] *= y[i];
33     fft(x, 1);
34     vector<ll> ret(sz);
35     for(int i = 0; i < sz; ++i) ret[i] = (ll)round(x[i].real());
36     while((int)ret.size() > 1 && ret.back() == 0) ret.pop_back();
37     return ret;
38 }
```

7.2 FWDT

```
1 const int MOD = 1e9+7;
2 const int inv2 = (MOD + 1) >> 1;
3 #define M (1 << 20)
4 #define OR 0
5 #define AND 1
6 #define XOR 2
7
8 int P1[M],P2[M];
9 void wt(int *a, int n, int flag = XOR){
10     if (n == 0) return;
11     int m = n / 2;
12     wt(a, m, flag); wt(a + m, m, flag);
13     for(int i = 0; i < m; i++){
14         int x = a[i], y = a[i + m];
15         if (flag == OR) a[i] = x, a[i + m] = (x + y) % MOD;
16         if (flag == AND) a[i] = (x + y) % MOD, a[i + m] = y;
17         if (flag == XOR) a[i] = (x + y) % MOD, a[i + m] = (x - y + MOD
18             ) % MOD;
19     }
20 }
21 void iwt(int* a, int n, int flag = XOR) {
22     if (n == 0) return;
23     int m = n / 2;
24     iwt(a, m, flag); iwt(a + m, m, flag);
25     for(int i = 0; i < m; i++){
26         int x = a[i], y = a[i + m];
27         if (flag == OR) a[i] = x, a[i + m] = (y - x + MOD) % MOD;
28         if (flag == AND) a[i] = (x - y + MOD) % MOD, a[i + m] = y;
29         if (flag == XOR) a[i] = 1LL * (x + y) * inv2 % MOD, a[i + m] =
30             1LL * (x - y + MOD) * inv2 % MOD; // replace inv2 by >>1 if
31             not required
32     }
33 }
34 vector<int> multiply(int n, vector<int> &A, vector<int> &B, int
35     flag = XOR) {
36     assert(__builtin_popcount(n) == 1);
37     A.resize(n); B.resize(n);
38     for(int i = 0; i < n; i++) P1[i] = A[i];
39     for(int i = 0; i < n; i++) P2[i] = B[i];
40     wt(P1, n, flag); wt(P2, n, flag);
41     for (int i = 0; i < n; i++) P1[i] = 1LL * P1[i] * P2[i] % MOD;
42     iwt(P1, n, flag);
43     return vector<int> (P1, P1 + n);
44 }
```

7.3 Xor Basis

```
1 template<typename T = int, int B = 31>
2 struct Basis{
3     T basis[B]; int sz;
4     void clear() {
```

```

5     memset(basis, 0, sizeof basis);
6     sz = 0;
7 }
8 Basis(){clear();}
9
10 void insert(T x) {
11     for(int i=B-1;i>=0;i--){
12         if(x>>i&1){
13             if(basis[i])x ^= basis[i];
14             else{
15                 basis[i] = x;
16                 sz++;
17                 break;
18             }
19         }
20     }
21 }
22
23 bool can(T x){
24     for(int i=B-1;i>=0;i--){
25         x = min(x, x ^ basis[i]);
26         return x == 0;
27     }
28
29 T kth(T k){
30     T x = 0;
31     T cnt = ((T)1<sz);
32     for(int i=B-1;i>=0;i--){
33         if(!basis[i])continue;
34         if(k>(cnt>>1)){
35             if (!(x>>i&1))x ^= basis[i];
36             k -= (cnt>>1);
37         }else{
38             if (x>>i&1)x ^= basis[i];
39         }
40         cnt>>=1;
41     }
42     return x;
43 }
44
45 bool merge(Basis &b){
46     bool flag = false;
47     for(ll i=B-1;i>=0;i--){
48         if(b.basis[i] && !insert(b.basis[i])) flag = true;
49     }
50     return flag;
51 }
52 };

```

7.4 Matrix

```

1 struct Mat {
2     int n, m;
3     vector<vector<int>> a;
4     Mat() { }
5     Mat(int _n, int _m) {n = _n; m = _m; a.assign(n, vector<int>(m, 0)); }
6     Mat(vector< vector<int> > v) { n = v.size(); m = n ? v[0].size() : 0; a = v; }
7     inline void make_unit() {
8         assert(n == m);
9         for (int i = 0; i < n; i++) {
10             for (int j = 0; j < n; j++) a[i][j] = i == j;
11         }
12     }
13     inline Mat operator + (const Mat &b) {
14         assert(n == b.n && m == b.m);
15         Mat ans = Mat(n, m);

```

```

16         for(int i = 0; i < n; i++) {
17             for(int j = 0; j < m; j++) {
18                 ans.a[i][j] = (a[i][j] + b.a[i][j]) % mod;
19             }
20         }
21         return ans;
22     }
23     inline Mat operator - (const Mat &b) {
24         assert(n == b.n && m == b.m);
25         Mat ans = Mat(n, m);
26         for(int i = 0; i < n; i++) {
27             for(int j = 0; j < m; j++) {
28                 ans.a[i][j] = (a[i][j] - b.a[i][j] + mod) % mod;
29             }
30         }
31         return ans;
32     }
33     inline Mat operator * (const Mat &b) {
34         assert(m == b.n);
35         Mat ans = Mat(n, b.m);
36         for(int i = 0; i < n; i++) {
37             for(int j = 0; j < b.m; j++) {
38                 for(int k = 0; k < m; k++) {
39                     ans.a[i][j] = (ans.a[i][j] + 1LL * a[i][k] * b.a[k][j] %
40                         mod) % mod;
41                 }
42             }
43             return ans;
44         }
45         inline Mat pow(long long k) {
46             assert(n == m);
47             Mat ans(n, n), t = a; ans.make_unit();
48             while (k) {
49                 if (k & 1) ans = ans * t;
50                 t = t * t;
51                 k >>= 1;
52             }
53             return ans;
54         }
55         inline Mat& operator += (const Mat& b) { return *this = (*this)
56             + b; }
57         inline Mat& operator -= (const Mat& b) { return *this = (*this)
58             - b; }
59         inline Mat& operator *= (const Mat& b) { return *this = (*this)
60             * b; }
61         inline bool operator == (const Mat& b) { return a == b.a; }
62         inline bool operator != (const Mat& b) { return a != b.a; }
63     };
64
65 // Usage
66 // Mat a(n, n);
67 // Mat b(n, n);
68 // Mat c = a * b;
69 // Mat d = a + b;
70 // Mat e = a - b;
71 // Mat f = a.pow(k);
72 // a.a[i][j] = x;

```

8 Strings

8.1 Aho Corasick

```

1 struct AC{
2     static const int K = 26;
3     struct Node{
4         int ch[K];
5         int fail;

```

```

6         int last_m;
7         vi id;
8         Node(){
9             memset(ch,0,sizeof(ch));
10            fail = last_m = 0;
11        }
12    };
13
14    vector<Node> t;
15
16    AC(){t.emplace_back();}
17
18    void insert(string& s, int id){
19        int u = 0;
20        for(char c:s){
21            int v = c-'a';
22            if(!t[u].ch[v]){
23                t[u].ch[v] = sz(t);
24                t.emplace_back();
25            }
26            u = t[u].ch[v];
27        }
28        t[u].id.push_back(id);
29    }
30
31    void build(){
32        queue<int> q;
33        for(int i=0;i<K;i++)
34            if (t[0].ch[i]) q.push(t[0].ch[i]);
35
36        while(!q.empty()){
37            int u=q.front();q.pop();
38            int f = t[u].fail;
39            t[u].last_m = t[f].id.empty() ? t[f].last_m:f;
40
41            for(int i=0;i<K;i++){
42                int &v = t[u].ch[i];
43                if(v){
44                    t[v].fail = t[f].ch[i];
45                    q.push(v);
46                }else v = t[f].ch[i];
47            }
48        }
49    }
50
51    int count(string &s){
52        ll ans = 0;
53        int u = 0;
54        for(char c:s){
55            u = t[u].ch[c - 'a'];
56            int temp = u;
57            while(temp > 0){
58                ans += t[temp].id.size();
59                temp = t[temp].last_m;
60            }
61        }
62        return ans;
63    }
64 };

```

8.2 Dynamic Aho Corasick

```

1 vector<AC> a(19);// log aho corasick
2 vector<vector<string>> ws;
3 int cnt = 0;//number of strings insert
4
5 auto insert = [&](string &s){
6     int pos = 0;

```



```

7   for(int i=0;i<19;i++)
8       if(ws[i].empty()){
9           pos = i;
10          break;
11      }
12
13  ws[pos].pb(s);
14  for(int i=0;i<pos;i++){
15      for(auto &str:ws[i])
16          ws[pos].pb(move(str));
17      ws[i].clear();
18      w[i] = AC();
19  }
20
21  w[pos] = AC();
22
23  for(int i=0;i<sz(ws[pos]);i++)w[pos].insert(ws[pos][i],i+1);
24
25  w[pos].build();
26
27  };
28
29  auto query = [&](string &s)->ll{
30      ll ans = 0;
31      for(int i=0;i<19;i++)
32          if(cnt>>i&1)
33              ans += a[i].count(s);
34  };

```

8.3 Suffix Array

```

1  struct suffix{
2      int index;
3      pii rank;
4      bool operator<(suffix b){return rank<b.rank;}
5      bool operator>(suffix b){return rank>b.rank;}
6  };
7
8  class Suffix_Array{
9  public:
10     string s;int n;
11     vector<suffix> suffixes;
12     vi lcp;
13     Suffix_Array(string x){
14         s = x+"$";
15         n = sz(s);
16         suffixes.resize(n);
17     }
18
19
20     void radix_sort(vector<suffix> &suff){
21         for(int i: vi{2,1}){
22             auto key = [&](const suffix &x){
23                 return i == 1?x.rank.F:x.rank.S;
24             };
25
26             int mx = 0;
27             for (const auto &i:suff) { mx = max(mx, key(i)); }
28             vector<int> occs(mx + 1);
29             for (const auto &i:suff)occs[key(i)]++;
30             vector<int> start(mx + 1);
31             for (int i=1;i<mx; i++) start[i] = start[i-1]+occs[i-1];
32
33             vector<suffix> new_arr(suff.size());
34             for (const auto &i : suff) {
35                 new_arr[start[key(i)]] = i;
36                 start[key(i)]++;
37             }

```

```

38         suff = new_arr;
39     }
40 }
41
42 void build(){
43     for(int i=0;i<n;i++)suffixes[i].index = i , suffixes[i].rank =
44         {s[i],(i+1<n?s[i+1]:-1)};
45     sort(all(suffixes));
46     vi equiv(n);
47
48     for(int i=1;i<n;i++){
49         auto c_val = suffixes[i].rank;
50         int cs = suffixes[i].index;
51         auto p_val = suffixes[i - 1].rank;
52         int ps = suffixes[i-1].index;
53         equiv[cs] = equiv[ps] + (c_val > p_val);
54     }
55
56     for(int cmp_ant = 1;cmp_ant <n; cmp_ant<=<=1){
57         for(auto &x:suffixes)
58             x.rank = {equiv[x.index],equiv[(x.index+cmp_ant)%n]};
59
60         //Std sort O(nlognlogn) Radix Sort O(nlogn)
61         radix_sort(suffixes);
62
63         for(int i=1;i<n;i++){
64             auto c_val = suffixes[i].rank;
65             int cs = suffixes[i].index;
66             auto p_val = suffixes[i - 1].rank;
67             int ps = suffixes[i-1].index;
68             equiv[cs] = equiv[ps] + (c_val > p_val);
69         }
70     }
71
72     void build_lcp(){
73         vector<int> suff_ind(n);
74         for (int i = 0; i < n; i++)suff_ind[suffixes[i].index] = i;
75         lcp.resize(n-1);
76         int start_at = 0;
77         for (int i = 0; i < n - 1; i++) {
78             int prev = suffixes[suff_ind[i] - 1].index;
79             int curr_cmp = start_at;
80             while (s[i + curr_cmp] == s[prev + curr_cmp])curr_cmp++;
81             lcp[suff_ind[i] - 1] = curr_cmp;
82             start_at = max(curr_cmp - 1, 0);
83         }
84     }
85
86     void show(){
87         for(int i=0;i<n;i++)cout << suffixes[i].index << " \n"[(i+1)==
88             n];
89     };

```

8.4 Hashing

```

1  ll binpow(ll n,ll k,const int mod){
2      ll res = 1;
3      n%=mod;
4      while(k){
5          if(k&1)(res*=n)%=mod;
6          (n*=n)%=mod;
7          k>>=1;
8      }
9      return res;
10 }
11

```

```

12 const int MOD1 = 127657753,MOD2 = 987654319;
13 const int P1 = 137,P2 = 277;
14 const int MAX = 1e6+9;
15
16 vector<pll> P(MAX),I_P(MAX);
17
18 void calc(){
19     P[0] = {1,1};
20     for(int i=1;i<MAX;i++){
21         P[i].F = P[i-1].F*P1%MOD1;
22         P[i].S = P[i-1].S*P2%MOD2;
23     }
24     int i1 = binpow(P1,MOD1-2,MOD1);
25     int i2 = binpow(P2,MOD2-2,MOD2);
26     I_P[0] = {1,1};
27     for(int i=1;i<MAX;i++){
28         I_P[i].F = I_P[i-1].F*i1%MOD1;
29         I_P[i].S = I_P[i-1].S*i2%MOD2;
30     }
31 }
32
33 struct Hashing {
34     int n;
35     string s; // 0 - indexed
36     vector<pll> hs; // 1 - indexed
37
38     Hashing(string _s) {
39         n = _s.size();
40         s = _s;
41         hs.emplace_back(0, 0);
42         for (int i = 0; i < n; i++) {
43             pll p;
44             p.F = (hs[i].F + P[i].F * (s[i]-'a'+1) % MOD1) % MOD1;
45             p.S = (hs[i].S + P[i].S * (s[i]-'a'+1) % MOD2) % MOD2;
46             hs.push_back(p);
47         }
48     }
49
50     pll get_hash(int l, int r) { // 1 - indexed
51         pll ans;
52         ans.F = (hs[r].F - hs[l - 1].F+ MOD1) * 1LL * I_P[l - 1].F%
53             MOD1;
54         ans.S = (hs[r].S - hs[l - 1].S+ MOD2) * 1LL * I_P[l - 1].S%
55             MOD2;
56         return ans;
57     }
58 };

```

9 Dp

9.1 Convex Hull Trick

```

1  struct CHT {
2      vector<ll> m, b;
3      int ptr = 0;
4
5      bool bad(int l1, int l2, int l3) {
6          return 1.0*(b[l3]-b[l1])*(m[l1]-m[l2])<=1.0*(b[l2]-b[l1])*(m[
7              l1]-m[l3]); //(slope dec+query min), (slope inc+query max)
8          return 1.0*(b[l3]-b[l1])*(m[l1]-m[l2]) >1.0*(b[l2]-b[l1])*(m[
9              l1]-m[l3]); //(slope dec+query max), (slope inc+query min)
10     }
11
12     void add(ll _m, ll _b){
13         m.push_back(_m);
14         b.push_back(_b);
15         int s = m.size();
16         while(s>=3&&bad(s - 3, s - 2, s - 1)){

```



```

15     s--;
16     m.erase(m.end()-2);
17     b.erase(b.end()-2);
18 }
19 }
20
21 ll f(int i, ll x){return m[i] * x + b[i];}
22
23 //(slope dec+query min), (slope inc+query max) -> x increasing
24 //(slope dec+query max), (slope inc+query min) -> x decreasing
25 ll query(ll x){
26     if(ptr >= sz(m))ptr = m.size() - 1;
27     while(ptr < sz(m)-1 && f(ptr + 1, x)<f(ptr, x))ptr++;
28     return f(ptr, x);
29 }
30
31 ll bs(int l, int r, ll x){
32     int mid = (l + r) / 2;
33     if(mid + 1 < m.size() && f(mid + 1, x) < f(mid, x)) return bs(
34         mid + 1, r, x); // > for max
35     if(mid - 1 >= 0 && f(mid - 1, x) < f(mid, x)) return bs(l, mid
36         - 1, x); // > for max
37     return f(mid, x);
38 }
39 };

```

9.2 Knuth Division

```

1  int N;
2  // read N and input
3  int dp[N][N], opt[N][N];
4
5  auto C = [&](int i, int j) {
6      // Implement cost function C.
7  };
8
9  for (int i=0;i<N;i++){
10     opt[i][i] = i;
11     // Initialize dp[i][i] according to the problem
12 }
13
14 for (int i = N-2; i >= 0; i--){
15     for (int j = i+1; j < N; j++){
16         int mn = INT_MAX;
17         int cost = C(i, j);
18         for (int k = opt[i][j-1]; k <= min(j-1, opt[i+1][j]); k++) {
19             if (mn >= dp[i][k] + dp[k+1][j] + cost) {
20                 opt[i][j] = k;
21                 mn = dp[i][k] + dp[k+1][j] + cost;
22             }
23         }
24         dp[i][j] = mn;
25     }
26 }
27 ans = dp[0][N-1];

```

9.3 Li Chao Tree

```

1  typedef ll ftype;
2  typedef complex<ftype> point;
3  #define x real
4  #define y imag
5  const ll MOD=1e9+7; //998244353LL;
6  const ll INF=1LL<<60;
7  const int maxn = 2e5;
8  point line[4 * maxn];
9  ftype dot(point a, point b) {
10     return (conj(a) * b).x();

```

```

11 }
12 ftype f(point a, ftype x) {
13     return dot(a, {x, 1});
14 }
15 void add_line(point nw, int v, int l, int r, int
16     lf, int rf) {
17     int m = (l + r) / 2;
18     if(rf < l || r <= lf) return;
19     if(l < lf || rf < r-1){
20         add_line(nw, 2 * v, l, m, lf, rf);
21         add_line(nw, 2 * v + 1, m, r, lf, rf);
22     }
23     bool lef = f(nw, l) < f(line[v], l);
24     bool mid = f(nw, m) < f(line[v], m);
25     if(!mid) {
26         swap(line[v], nw);
27     }
28     if(r - l == 1) {
29         return;
30     }
31     else if(lef != mid) {
32         add_line(nw, 2 * v, l, m, lf, rf);
33     }
34     else {
35         add_line(nw, 2 * v + 1, m, r, lf, rf);
36     }
37 }
38 ftype get(int x, int v, int l, int r) {
39     int m = (l + r) / 2;
40     if(r - l == 1) {
41         return f(line[v], x);
42     }
43     else if(x < m) {
44         return max(f(line[v], x), get(x, 2 * v,
45             l, m));
46     }
47     else {
48         return max(f(line[v], x), get(x, 2 * v +
49             1, m, r));
50     }

```

10 Miscellaneous

10.1 Rnd template

```

1  mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count()
2      );

```